

Санкт-Петербургский государственный университет
Факультет прикладной математики – процессов управления
ООП СВ.5005.2022 Прикладная математика, фундаментальная информатика и программирование

Шаго Павел Евгеньевич

**Математическое моделирование и реализация планировщика
процессов для гетерогенных процессорных архитектур**

Бакалаврская работа

Научный руководитель:
кандидат физ.-мат. наук, доцент
Корхов В. В.

Рецензент:
эксперт по архитектуре и сетевому стеку,
ООО НИЦ АЭРОДИСК, Анохин Ю. В.

Санкт-Петербург
2026

- Современные процессоры всё чаще имеют гетерогенную компоновку, в которой производительные P-ядра дополняются энергоэффективными E-ядрами
- Стандартный планировщик Linux не различает типы ядер, поэтому не всегда способен сбалансировать производительность и энергопотребление так, как это задумано архитектурой CPU

Цели работы

- 1 Формализовать распределение временных слотов P/E-ядер как динамический аукцион Викри (VCG), в котором задачи конкурируют за вычислительный ресурс
- 2 Реализовать полученный механизм используя `sched_ext` и оценить его по метрикам на тестах производительности

Каждое ядро рассматривается как **истощаемое благо**, неиспользованный квант времени теряется безвозвратно. Задача-агент характеризуется типом $\theta_i = (v_i, l_i)$, а механизм максимизирует социальное благосостояние.

Аппаратная платформа

- Ядра: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$,
 $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$
- Дискретное время $t = 1, 2, 3, \dots$
- Замедление на E-ядре:
 $\sigma = \eta_P / \eta_E > 1$
- Общий ISA (любая задача может быть выполнена на любом ядре)

Задачи как агенты

- Тип $\theta_i = (v_i, l_i)$ (оценка и длина)
- Полезность $u_i = v_i - p_i$ при полном выполнении
- Бюджет B_i по классу обслуживания:

Реального времени	$B_{RT} \gg 1$
Интерактивный	B_{int}
Пакетный	B_{batch}
Фоновый	$B_{bg} \approx 0$

На E-ядре задача длиной l_i исполняется за $\lceil l_i \cdot \sigma \rceil$ квантов. Ценность v_i начисляется только при полном выполнении (аналог контракта Сано).

Состояние $s^t = (\mathbf{x}^t, \mathbf{b}^t, \omega^t)$

- $\mathbf{x}^t$ – оставшиеся кванты на каждом ядре
- $\mathbf{b}^t$ – остатки бюджетов активных задач
- ω^t – внешние факторы (температура, нагрузка)

Действие $a^t = (a_{i,P}^t, a_{i,E}^t)_i$ при ограничениях:

$$a_{i,P}^t + a_{i,E}^t \leq 1, \quad \forall i$$

$$\sum_i a_{i,P}^t \leq K_P^t, \quad \sum_i a_{i,E}^t \leq K_E^t$$

Вознаграждение

Задача i (покупатель):

$$r_i(s, a) = v_i \cdot \mathbb{1}[i \text{ получила ядро}]$$

Планировщик (продавец):

$$r_0 = \mu_P \sum_{k \in \mathcal{P}} \mathbb{1}[x_k \geq 1] + \mu_E \sum_{k \in \mathcal{E}} \mathbb{1}[x_k \geq 1]$$

Социальное благосостояние

$$W(\pi) = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T R(s^t, a^t) \right]$$

Эффективная ценность задачи i на ядре типа $\kappa \in \{P, E\}$

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \sigma \rceil$$

Оптимальное распределение (максимизация W)

$$a^t = \arg \max_a \sum_i [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \mathbb{E}[W(s^{t+1})]$$

Платёж VCG задачи i

$$p_i^t = W_{-i}(s^t) - (W(s^t) - \phi_\kappa(\theta_i))$$

Задача i платит ровно то, что теряют остальные агенты из-за её присутствия (pivot rule)

Свойства механизма

- Стимульная совместимость (выгодно сообщать истинный θ_i)
- Индивидуальная рациональность $u_i \geq 0$
- Бюджетное ограничение $p_i \leq B_i^t$

Прокси для распределения: отношение бюджета $\rho_i = B_i / B_i^{\max}$

DSQ_P

$$\rho_i \geq \rho^* (50\%)$$

Интерактивные,
«bursty» задачи

Высокий ρ_i = много
простоя = чувствитель-
ность к задержке

DSQ_E

$$\rho_i < \rho^*$$

CPU-bound,

пакетные задачи

Низкий ρ_i = активно
работает = подходит
для E-ядра

DSQ_Starved

$$B_i < B_i^{\max} / 10$$

Бюджет исчерпан

$$v_d += \max(0, -\phi_\kappa) / D$$

Бюджет пополняется пропорционально времени простоя: $B_i += \Delta t_{\text{idle}} \cdot v_i / D_{\text{rep}}$ —аналог
порогового правила в одномерных аукционах

Отклонения от абстрактной модели:

(1) Непрерывная формула ϕ (нс, не кванты):

$$\phi_P = v_i \cdot s - c_P \cdot l_{ns}, \quad \phi_E = v_i \cdot s - c_E \cdot l_{ns} \cdot \sigma$$

где l_{ns} —EWMA измеренного времени исполнения; $\phi < 0$ означает прямое направление в DSQ_Starved

(2) Пропорциональный платёж:

$$p_i = c_{\kappa} \cdot t_{\text{consumed}} / s$$

устраняет штраф за первый квант у коротких задач

Параметры

C_P_DEF	512
C_E_DEF	256
BURST_PCT	50%
BUDGET_MUL	2000
STARVE_FRAC	10
P_CAP_PCT	90%

Система

Ядро:
Linux 7.0

CPU:
Intel Core
Ultra 7 265K
12E + 8P

Сравнение

Linux EEVDF
vs
scx_auction
(VCG)

Бенчмарки

- **hackbench** —1000 задач
Интенсивный обмен сообщениями,
высокая нагрузка на планировщик.
Оценивает переключение контекста.
- **sysbench** —стандартные параметры
CPU, память, дисковый I/O.
Моделирует OLTP под реалистичной
нагрузкой.
- **sched-latency** —собственный eBPF
Трейсинг трейспоинтов, запись p99
задержки планирования.

Метрики

hackbench
Время (s)

sysbench
Events/s

sched-latency
p99 (μs)

Нет данных

Замеры `scx_auction` в процессе сбора

Метрика: полное время выполнения (s), меньше — лучше

Нагрузка: 1000 задач, интенсивный обмен сообщениями

Ожидание: нейтральный результат — VCG-маршрутизация при высокой конкуренции сводится к базовому EEVDF

Нет данных

Замеры `scx_auction` в процессе сбора

Метрика: пропускная способность (events/s), больше —лучше

Нагрузка: CPU-bound, однопоточная вычислительная задача

Ожидание: задача с низким ρ_i маршрутизируется на E-ядро; потенциальный выигрыш при длинном фиксированном кванте

Нет данных

Замеры `scx_auction` в процессе сбора

Метрика: p99 задержка планирования (μs), меньше — лучше

Нагрузка: трейсинг трейспоинтов под смешанной нагрузкой

Ожидание: интерактивные задачи (высокий ρ_i) на P-ядре — потенциальное снижение хвостовой задержки относительно однородного планировщика

Нет данных

Анализ будет проведён после сбора результатов

Ключевые вопросы:

- Корректно ли ρ_i разделяет интерактивные и CPU-bound задачи?
- Насколько велик overhead VCG-платежей относительно ядерного EEVDF?
- Подтверждается ли снижение хвостовой задержки для P-ядерных задач?

- Задача распределения P/E-ядер формализована как динамический VCG-аукцион в рамках MDP
- Механизм реализован через sched_ext (eBPF) —политика изменяется без пересборки ядра
- Маршрутизация через отношение бюджета ρ_i —практическое приближение к оптимальному пороговому правилу аукциона
- Направления развития: онлайн-обучение (IHMDP-VCG), сравнение с ядерным EEVDF и s3+ (capacity-aware)

- [1] Leon G., Etesami O. Mechanism Design for Scheduling with Heterogeneous Workers. Advances in Neural Information Processing Systems, 2022
- [2] Sano T. Dynamic Mechanism Design for Multi-Period Contracting. Working paper, 2023
- [3] The extensible scheduler class (LWN)
- [4] An EEVDF CPU scheduler for Linux (LWN)

Стимульная совместимость (Sano, Теорема 1)

Механизм стимульно совместим тогда и только тогда, когда для каждой задачи i :

- 1 $\alpha_i(v_i, l_i)$ не убывает по v_i при фиксированном l_i
- 2 $\int_0^{v_i} \alpha_i(\nu, l_i) d\nu$ не возрастает по l_i при фиксированном v_i
- 3 Существует $\underline{\Pi}_i$, не зависящее от l_i :

$$\Pi_i(v_i, l_i) = \underline{\Pi}_i + \int_0^{v_i} \alpha_i(\nu, l_i) d\nu$$

Задачам невыгодно завышать оценку v_i или занижать длину l_i —правдивая ставка является доминирующей стратегией

Онлайн-обучение: IHMDP-VCG (Leon & Etesami)

При неизвестных распределениях типов задач и ядре MDP планировщик обучается онлайн. Эпизодическая структура:

- **Фаза перемешивания** (длина $d^{[k]}$): вывод цепи Маркова на стационарное распределение
- **Стационарная фаза** (длина $l^{[k]}$): сбор платежей $\hat{p}^{[k]}$, обновление оценок

Гарантии при $T \rightarrow \infty$ (с вер. $\geq 1 - O(\zeta)$):

Регрет благосостояния	$\tilde{O}(\varepsilon T n + T^{2/3} n)$
Регрет продавца	$\tilde{O}(\varepsilon T n^2 + T^{2/3} n^2)$
Регрет покупателей	$\tilde{O}(\varepsilon T n^2 + T^{2/3} n^2)$

Асимптотически:
 стимульная индивидуальная одновременно
 эффективность, совместимость и рациональность —

Пример: $K_P = K_E = 1$, одно свободное ядро типа κ

Победитель:

$$i^* = \arg \max_{i \in \mathcal{N}^t, B_i^t > 0} \phi_\kappa(\theta_i) + \delta^{m_\kappa(l_i)} \bar{W}_\kappa$$

где $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$

Платёж победителя:

$$p_{i^*} = \phi_\kappa^{-1} \left(\phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa \mid l_{i^*} \right)$$

где j — задача на втором месте.

$\phi_P(\theta_i) > \phi_E(\theta_i)$ выполняется не всегда: при малом v_i и большом l_i Е-ядро может дать большую эффективную ценность ($c_E \cdot \lceil l_i \sigma \rceil < c_P \cdot l_i$ при $\sigma < \gamma$). Оптимальная стратегия нетривиальна.